Scalable Public Transport Ticketing Data Pipeline and Analytics System

NAME : KAUSTAV MODAK
ROLL NO.:23052888
Data Engineering(IS30058)

Problem Statement

Public transport systems generate large volumes of transactional ticket data across multiple routes, cities, and vehicle types. This data is often stored in raw formats such as CSV files and lacks proper structure, validation, and analytical capability.
Key challenges include:

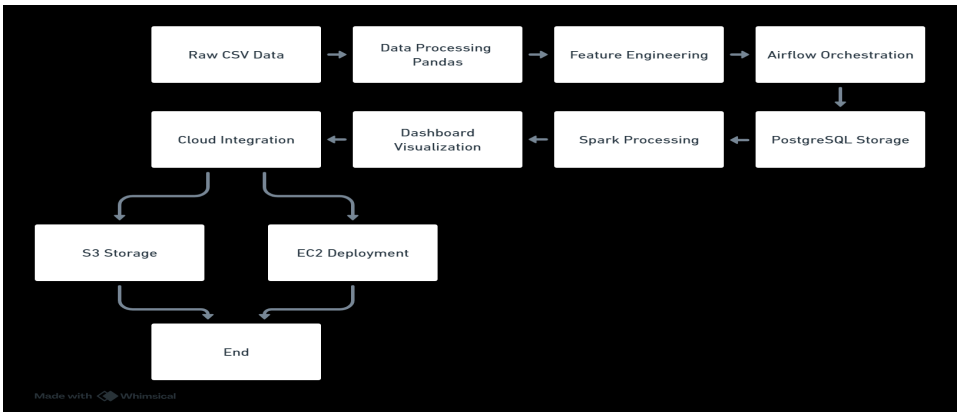
- Handling large-scale datasets (1M+ rows) efficiently
- Ensuring data quality (missing values, duplicates, anomalies)
- Integrating multiple data sources
- Supporting both batch and near real-time processing
- Providing actionable insights for decision-making

Traditional systems fail to provide a unified and automated solution that handles ingestion, processing, storage, and visualization in a scalable way.

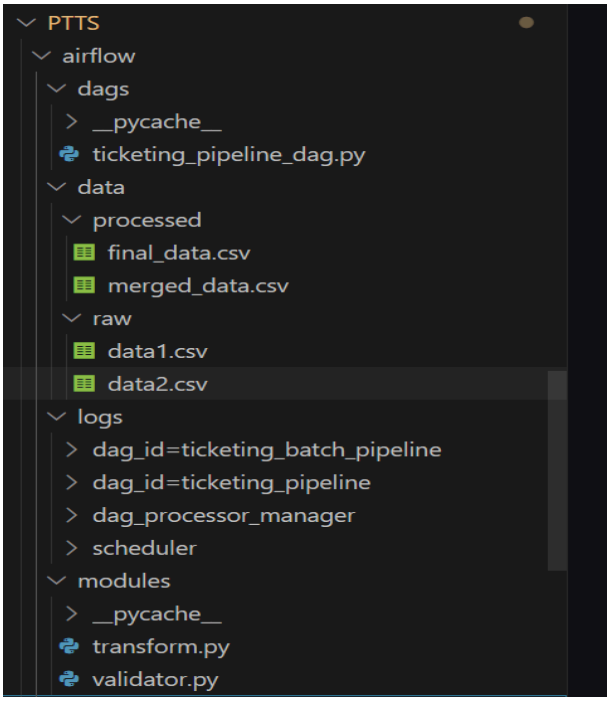
Proposed Solution

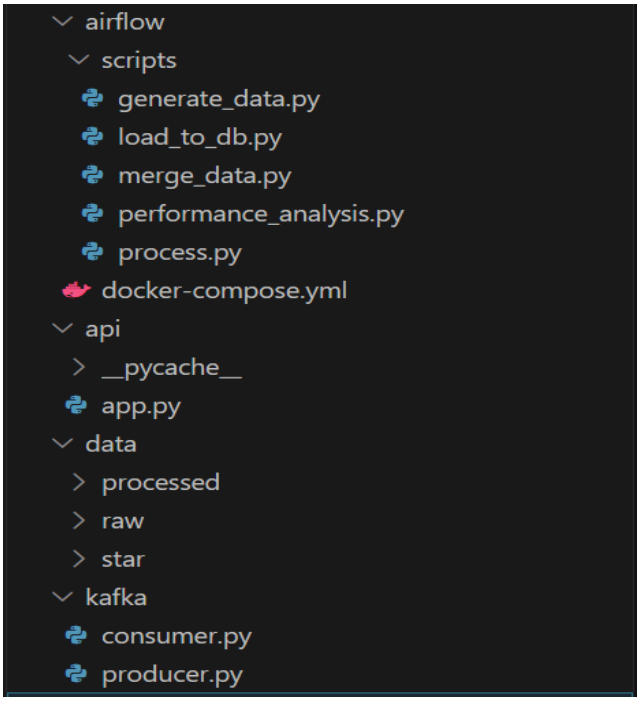
To address these challenges, an **end-to-end data engineering pipeline** was developed that automates the complete lifecycle of data — from ingestion to visualization.
The system is designed with modular components and integrates multiple technologies such as Airflow, Pandas, Spark, Kafka simulation, MySQL, PostgreSQL, AWS, and Streamlit.

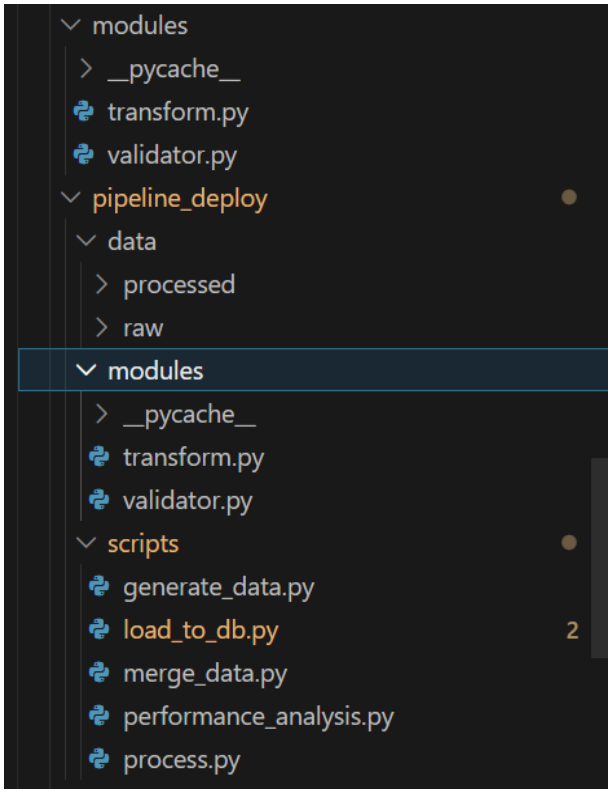
System Architecture Overview

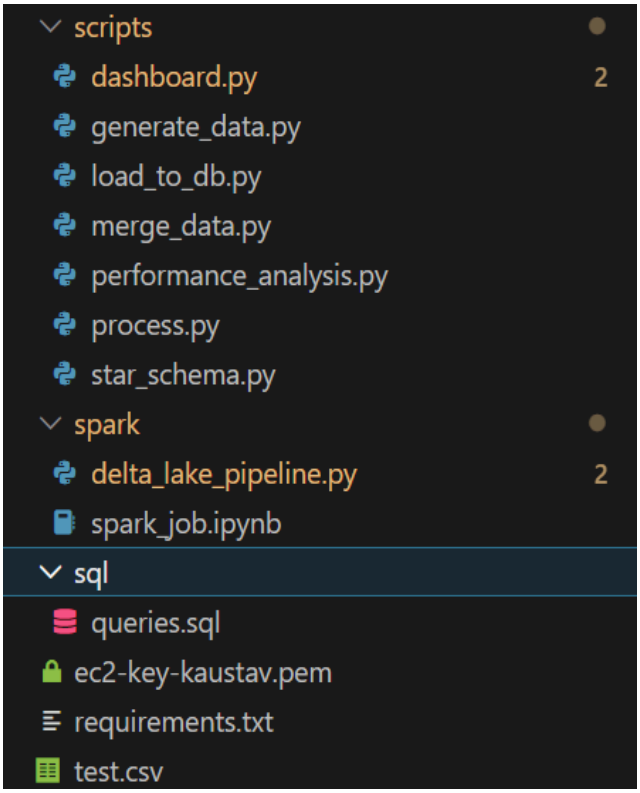


Project Directory Structure:









Implementation Details

1. Data Ingestion (Python + CSV)

- Multiple raw datasets (data1.csv, data2.csv) are generated and stored
- Data is read using Pandas
- Merging is handled using a custom script (merge_data.py)

Key Points:

- Handles multiple files
- Scalable for large datasets
- Structured ingestion process

```
ticket_id,user_id,route,price,city,vehicle_type
1,948,D-E,43,Delhi,Metro
2,591,C-D,74,Delhi,Metro
3,837,D-E,58,Mumbai,Metro
4,571,A-B,65,Delhi,Metro
5,996,C-D,90,Delhi,Metro
6,788,B-C,55,Kolkata,Bus
7,937,B-C,57,Kolkata,Metro
8,863,D-E,68,Kolkata,Metro
9,521,D-E,75,Kolkata,Metro
10,708,C-D,36,Mumbai,Metro
11,364,A-B,90,Kolkata,Metro
12,833,A-B,59,Kolkata,Bus
13,986,B-C,98,Kolkata,Metro
14,974,B-C,71,Mumbai,Train
15,130,A-B,32,Mumbai,Metro
16,239,C-D,94,Mumbai,Train
17,385,B-C,63,Mumbai,Metro
18,901,C-D,68,Kolkata,Bus
19,476,C-D,74,Delhi,Train
20,772,C-D,94,Mumbai,Bus
21,503,B-C,65,Kolkata,Metro
```

2. Data Processing & Transformation

Custom transformation module (transform.py) includes:

- Price normalization
- Feature engineering
- Aggregation functions

Processing pipeline (process.py) performs:

- Data loading
- Transformation
- Aggregation

```
ticket_id,user_id,route,price,city,vehicle_type,price_norm,price_category
1,948,D-E,43,Delhi,Metro,0.18840579710144928,Low
2,591,C-D,74,Delhi,Metro,0.6376811594202898,Medium
3,837,D-E,58,Mumbai,Metro,0.4057971014492754,Medium
4,571,A-B,65,Delhi,Metro,0.5072463768115942,Medium
5,996,C-D,90,Delhi,Metro,0.8695652173913043,High
6,788,B-C,55,Kolkata,Bus,0.36231884057971014,Medium
7,937,B-C,57,Kolkata,Metro,0.391304347826087,Medium
8,863,D-E,68,Kolkata,Metro,0.5507246376811594,Medium
9,521,D-E,75,Kolkata,Metro,0.6521739130434783,Medium
10,708,C-D,36,Mumbai,Metro,0.08695652173913043,Low
11,364,A-B,90,Kolkata,Metro,0.8695652173913043,High
12,833,A-B,59,Kolkata,Bus,0.42028985507246375,Medium
13,986,B-C,98,Kolkata,Metro,0.9855072463768116,High
14,974,B-C,71,Mumbai,Train,0.5942028985507246,Medium
15,130,A-B,32,Mumbai,Metro,0.028985507246376812,Low
16,239,C-D,94,Mumbai,Train,0.927536231884058,High
17,385,B-C,63,Mumbai,Metro,0.4782608695652174,Medium
18,901,C-D,68,Kolkata,Bus,0.5507246376811594,Medium
19,476,C-D,74,Delhi,Train,0.6376811594202898,Medium
20,772,C-D,94,Mumbai,Bus,0.927536231884058,High
21,503,B-C,65,Kolkata,Metro,0.5072463768115942,Medium
```

3. Data Validation & Quality Checks

Validation module (validator.py) includes:

- Missing value detection
- Duplicate record detection
- Anomaly detection

This ensures:

- Clean and reliable dataset
- Reduced data inconsistencies

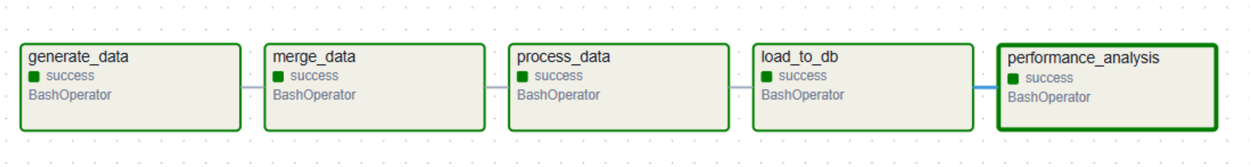
4. Workflow Orchestration using Airflow

An automated DAG (ticketing_pipeline_dag.py) orchestrates:

- Data generation
- Data merging
- Data processing
- Database loading
- Performance analysis

Features:

- Task scheduling
- Dependency management
- Retry mechanism

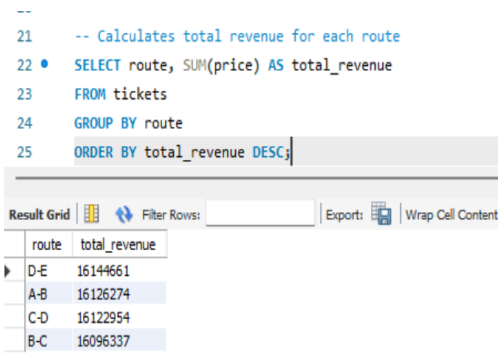


5. Database Integration (MySQL)

Processed data is stored in MySQLusing SQLAlchemy.
Implemented SQL operations:

- Aggregation (SUM, COUNT)
- Grouping (GROUP BY)
- Ranking (Window Functions)

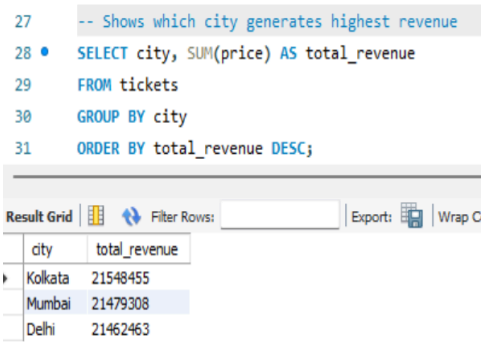
This enables efficient querying for analytics.



```
21  -- Calculates total revenue for each route
22  SELECT route, SUM(price) AS total_revenue
23  FROM tickets
24  GROUP BY route
25  ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

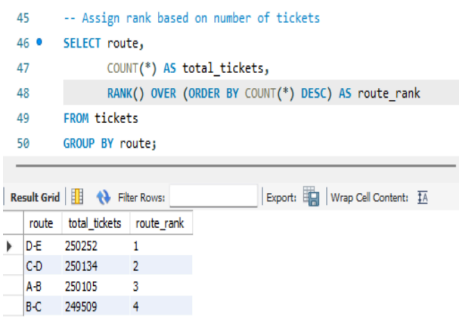
route	total_revenue
D-E	16144661
A-B	16126274
C-D	16122954
B-C	16096337



```
27  -- Shows which city generates highest revenue
28  SELECT city, SUM(price) AS total_revenue
29  FROM tickets
30  GROUP BY city
31  ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap C

city	total_revenue
Kolkata	21548455
Mumbai	21479308
Delhi	21462463



```
45  -- Assign rank based on number of tickets
46  SELECT route,
47         COUNT(*) AS total_tickets,
48         RANK() OVER (ORDER BY COUNT(*) DESC) AS route_rank
49  FROM tickets
50  GROUP BY route;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

route	total_tickets	route_rank
D-E	250252	1
C-D	250134	2
A-B	250105	3
B-C	249509	4

6. Performance Optimization (Pandas + NumPy)

- Processed dataset: 1 million records
- Memory optimization techniques applied
- Performance metrics tracked:
  - Load time
  - Processing time

- Memory usage

```
Memory BEFORE:
Index                 132
ticket_id         8000000
user_id           8000000
route            52000000
price             8000000
city             55000738
vehicle_type     53333490
price_norm        8000000
price_category   53571702
dtype: int64

Optimization Time: 1.0708942413330078
```

```
Memory AFTER:
Index                 132
ticket_id         8000000
user_id           8000000
route             1000380
price             8000000
city              1000273
vehicle_type      1000268
price_norm        8000000
price_category    1000268
dtype: int64

Aggregation Time: 0.11739087104797363
```

7. Big Data Processing (PySpark)

A Spark job (spark_job.ipynb) performs:

- Filtering large datasets
- Aggregations
- SQL-based queries
- Partitioning
- Caching

```
# ----------------------------------
# CREATE SPARK SESSION
# ----------------------------------
spark = SparkSession.builder \
    .appName("Transport Analysis") \
    .getOrCreate()

print("Spark Session Created")
```

```
Spark Session Created
```

```
# ----------------------------------
# LOAD DATA (1M rows)
# ----------------------------------

file_path = "../data/processed/final_data.csv"

df = spark.read.csv(
    file_path,
    header=True,
    inferSchema=True
)

print("Data Loaded")

df.show(5)
print("Total Rows:", df.count())
```

```
Data Loaded
+---------+-------+-----+-----+------+------------+-------------------+--------------+
|ticket_id|user_id|route|price|  city|vehicle_type|         price_norm|price_category|
+---------+-------+-----+-----+------+------------+-------------------+--------------+
|        1|    948|  D-E|   43| Delhi|       Metro|0.18840579710144928|           Low|
|        2|    591|  C-D|   74| Delhi|       Metro| 0.6376811594202898|        Medium|
|        3|    837|  D-E|   58|Mumbai|       Metro| 0.40579710144492754|        Medium|
|        4|    571|  A-B|   65| Delhi|       Metro| 0.5072463768115942|        Medium|
|        5|    996|  C-D|   90| Delhi|       Metro| 0.8695652173913043|          High|
+---------+-------+-----+-----+------+------------+-------------------+--------------+
only showing top 5 rows

Total Rows: 1000000
```

```
Filtered Data:
+---------+-------+-----+-----+-------+------------+-------------------+--------------+
|ticket_id|user_id|route|price|   city|vehicle_type|         price_norm|price_category|
+---------+-------+-----+-----+-------+------------+-------------------+--------------+
|        2|    591|  C-D|   74|  Delhi|       Metro| 0.6376811594202898|        Medium|
|        3|    837|  D-E|   58| Mumbai|       Metro| 0.40579710144492754|        Medium|
|        4|    571|  A-B|   65|  Delhi|       Metro| 0.5072463768115942|        Medium|
|        5|    996|  C-D|   90|  Delhi|       Metro| 0.8695652173913043|          High|
|        6|    788|  B-C|   55|Kolkata|         Bus|0.36231884057971014|        Medium|
+---------+-------+-----+-----+-------+------------+-------------------+--------------+
only showing top 5 rows

Revenue by Route:
+-----+----------+
|route|sum(price)|
+-----+----------+
|  B-C|  16096337|
|  C-D|  16122954|
|  A-B|  16126274|
|  D-E|  16144661|
+-----+----------+

+-----+-----+
|route|price|
+-----+-----+
...
|  C-D|   90|
+-----+-----+
only showing top 5 rows
```

8. Streaming Pipeline (Kafka Simulation)

- Implemented producer-consumer model

- Simulates real-time data ingestion

Components:

- producer.py
- consumer.py

```
Sent to partition (by city=Delhi): {'ticket_id': 1, 'user_id': 948, 'route': 'D-E', 'price': 43, 'city': 'Delhi', 'vehicle_type': 'Metro', 'price_n
orm': 0.1884057971014492, 'price_category': 'Low'}
Sent to partition (by city=Delhi): {'ticket_id': 2, 'user_id': 591, 'route': 'C-D', 'price': 74, 'city': 'Delhi', 'vehicle_type': 'Metro', 'price_n
orm': 0.6376811594202898, 'price_category': 'Medium'}
Sent to partition (by city=Mumbai): {'ticket_id': 3, 'user_id': 837, 'route': 'D-E', 'price': 58, 'city': 'Mumbai', 'vehicle_type': 'Metro', 'price
_norm': 0.4057971014492754, 'price_category': 'Medium'}
Sent to partition (by city=Delhi): {'ticket_id': 4, 'user_id': 571, 'route': 'A-B', 'price': 65, 'city': 'Delhi', 'vehicle_type': 'Metro', 'price_n
orm': 0.5072463768115942, 'price_category': 'Medium'}
Sent to partition (by city=Delhi): {'ticket_id': 5, 'user_id': 996, 'route': 'C-D', 'price': 90, 'city': 'Delhi', 'vehicle_type': 'Metro', 'price_n
orm': 0.8695652173913043, 'price_category': 'High'}
Sent to partition (by city=Kolkata): {'ticket_id': 6, 'user_id': 788, 'route': 'B-C', 'price': 55, 'city': 'Kolkata', 'vehicle_type': 'Bus', 'price
_norm': 0.3623188405797101, 'price_category': 'Medium'}
Sent to partition (by city=Kolkata): {'ticket_id': 7, 'user_id': 937, 'route': 'B-C', 'price': 57, 'city': 'Kolkata', 'vehicle_type': 'Metro', 'pri
ce_norm': 0.391304347826087, 'price_category': 'Medium'}
Sent to partition (by city=Kolkata): {'ticket_id': 8, 'user_id': 863, 'route': 'D-E', 'price': 68, 'city': 'Kolkata', 'vehicle_type': 'Metro', 'pri
ce_norm': 0.5507246376811594, 'price_category': 'Medium'}
Sent to partition (by city=Kolkata): {'ticket_id': 9, 'user_id': 521, 'route': 'D-E', 'price': 75, 'city': 'Kolkata', 'vehicle_type': 'Metro', 'pri
ce_norm': 0.6521739130434783, 'price_category': 'Medium'}
Sent to partition (by city=Mumbai): {'ticket_id': 10, 'user_id': 708, 'route': 'C-D', 'price': 36, 'city': 'Mumbai', 'vehicle_type': 'Metro', 'pric
e_norm': 0.0869565217391304, 'price_category': 'Low'}
Sent to partition (by city=Kolkata): {'ticket_id': 11, 'user_id': 364, 'route': 'A-B', 'price': 90, 'city': 'Kolkata', 'vehicle_type': 'Metro', 'pr
ice_norm': 0.8695652173913043, 'price_category': 'High'}
```

```
Received:
Partition: 0
Offset: 0
Key: Delhi
Value: {'ticket_id': 1, 'user_id': 948, 'route': 'D-E', 'price': 43, 'city': 'Delhi', 'vehicle_type': 'Metro', 'price_norm': 0.1884057971014492,
rice_category': 'Low'}


Received:
Partition: 0
Offset: 1
Key: Delhi
Value: {'ticket_id': 2, 'user_id': 591, 'route': 'C-D', 'price': 74, 'city': 'Delhi', 'vehicle_type': 'Metro', 'price_norm': 0.6376811594202898,
rice_category': 'Medium'}


Received:
Partition: 1
Offset: 0
Key: Mumbai
Value: {'ticket_id': 3, 'user_id': 837, 'route': 'D-E', 'price': 58, 'city': 'Mumbai', 'vehicle_type': 'Metro', 'price_norm': 0.4057971014492754,
price_category': 'Medium'}


Received:
Partition: 0
Offset: 2
Key: Delhi
Value: {'ticket_id': 4, 'user_id': 571, 'route': 'A-B', 'price': 65, 'city': 'Delhi', 'vehicle_type': 'Metro', 'price_norm': 0.5072463768115942,
```

9. Cloud Integration

AWS S3

- Uploaded dataset (final_data.csv)
- Verified download capability

```
C:\Users\KIIT0001\Desktop\DESKTOP_OWN\CODING\PTTS>aws configure
AWS Access Key ID [****************2G5P]:
AWS Secret Access Key [****************N/Ia]:
Default region name [ap-south-1]:
Default output format [json]:

C:\Users\KIIT0001\Desktop\DESKTOP_OWN\CODING\PTTS>aws s3 cp data/processed/final_data.csv s3://ticketing-data-kaustav/
upload: data\processed\final_data.csv to s3://ticketing-data-kaustav/final_data.csv
```

```
C:\Users\KIIT0001\Desktop\DESKTOP_OWN\CODING\PTTS>aws s3 cp s3://ticketing-data-kaustav/final_data.csv test.csv
download: s3://ticketing-data-kaustav/final_data.csv to .\test.csv
```

AWS EC2

- Deployed pipeline
- Executed scripts remotely
- Installed dependencies

```
(venv) ubuntu@ip-172-31-11-192:~/pipeline_deploy$ python scripts/generate_data.py
Two datasets created
(venv) ubuntu@ip-172-31-11-192:~/pipeline_deploy$ python scripts/merge_data.py
/home/ubuntu/pipeline_deploy/scripts/merge_data.py:11: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment u
sing an inplace method.
Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy
 (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' instead, to perform the operation inplace on the o
riginal object, or try to avoid an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html
  df['price'].fillna(df['price'].mean(), inplace=True)
Merged dataset (1M rows)
```

```
(venv) ubuntu@ip-172-31-11-192:~/pipeline_deploy$ python scripts/process.py

 Data Loaded Successfully (1M rows)

 Checking Null Values:
ticket_id       0
user_id         0
route           0
price           0
city            0
vehicle_type    0
dtype: int64

 Checking Duplicates:
(0, 6)

 Checking Anomalies:
(0, 6)

 Transformation Completed

 Revenue by Route:
route
A-B    16099891
B-C    16129049
C-D    16159984
D-E    16100452
Name: price, dtype: int64
```

```
(venv) ubuntu@ip-172-31-11-192:~/pipeline_deploy$ python scripts/load_to_db.py
Data loaded successfully
```

```
(venv) ubuntu@ip-172-31-11-192:~/pipeline_deploy$ python scripts/performance_analysis.py

 Time to load data: 0.9019064903259277 seconds

 Memory Usage BEFORE Optimization:
Index                 132
ticket_id         8000000
user_id           8000000
route            52000000
price             8000000
city             55000158
vehicle_type     53334372
price_norm        8000000
price_category   53572881
dtype: int64

 Optimization Time: 0.3346250057220459 seconds

 Memory Usage AFTER Optimization:
Index                 132
ticket_id         8000000
user_id           8000000
route             1000208
price             8000000
city              1000165
vehicle_type      1000160
price_norm        8000000
price_category    1000160
dtype: int64

 Aggregation Time (Optimized): 0.026164531707763672 seconds

 PERFORMANCE ANALYSIS COMPLETE
```

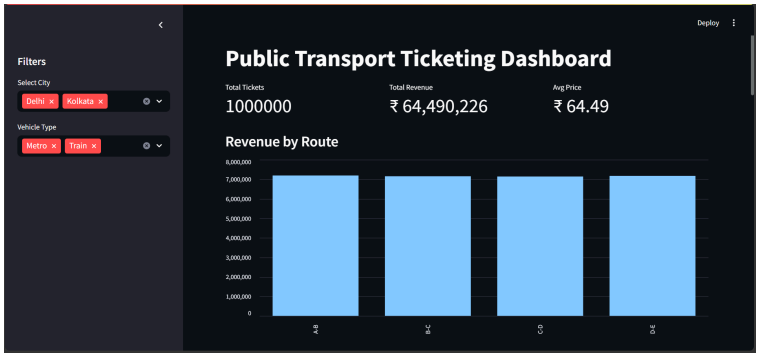
10. Dashboard Visualization (Streamlit)

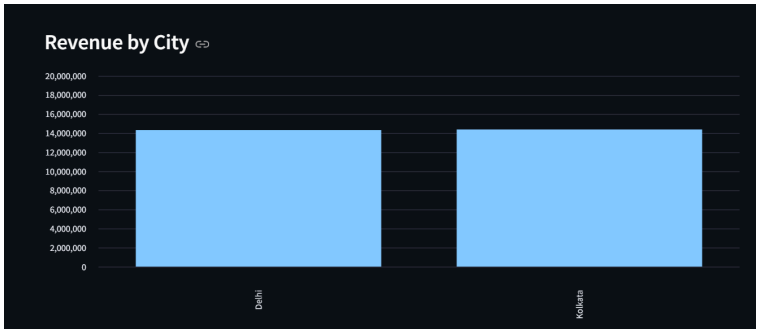
An interactive dashboard was built to visualize insights:

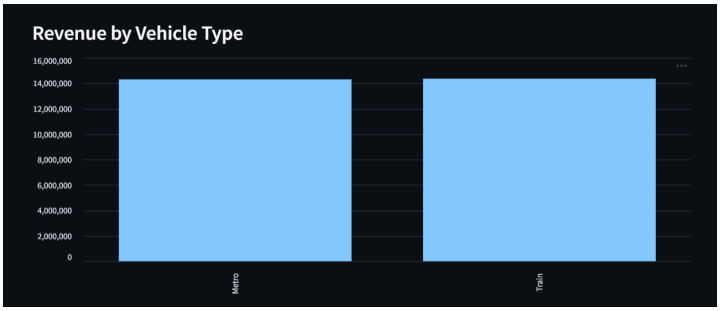
- Revenue by route
- Revenue by city
- Vehicle-wise revenue
- Price category distribution

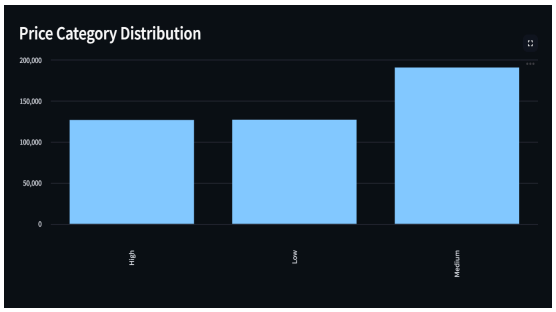
Features:

- Interactive filters
- Real-time updates
- Clean UI









Tech Stack

- **Languages**: Python
- **Libraries**: Pandas, NumPy
- **Orchestration**: Apache Airflow
- **Big Data**: Apache Spark
- **Streaming**: Kafka (simulation)
- **Database**: MySQL(for SQL queries), PostgreSQL(for cloud deployment)
- **Cloud**: AWS S3, EC2
- **Visualization**: Streamlit

Unique Contributions

- Built a **complete end-to-end data pipeline**
- Integrated **batch processing**
- Handled **large-scale dataset (1M+ rows)**
- Implemented **data validation system**
- Performed **cloud deployment (EC2 + S3)**
- **Designed modular and reusable architecture**

Future Improvements

- Real-time Spark Streaming with Kafka
- Integration with AWS Redshift / BigQuery
- Dashboard deployment on cloud
- ML-based demand prediction
- Automated alert system

Conclusion

This project successfully demonstrates a scalable and automated data pipeline capable of handling large datasets, ensuring data quality, and generating meaningful insights.
It integrates multiple modern data engineering tools and reflects real-world industry practices.